

BEST Coding Practices

Krzysztof Buzar

DSR 2025

Sources and Inspirations

- The Zen of Python
 - Tim Peters (2004)
- The Art of Computer Programming
 - Donald E. Knuth (1968 - Still incomplete!)
- Clean Code: A Handbook of Agile Software Craftsmanship
 - Robert C. Martin (2008)
- The Power of Ten – NASA/JPS Coding Guidelines
 - Rules for Developing Safety Critical Code
 - Gerard J. Holzmann (?)

The Zen of Python (PEP 20)

import this

The Zen of Python, by Tim Peters

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

The Zen of Python (PEP 20)

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

The Zen of Python (PEP 20)

There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.

Now is better than never.

Although never is often better than **right** now.

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those

Python – Best Practices

- Comments and Docstrings!
- Explicit Typing
- Variable naming conventions
- No **MAGIC NUMBERS**

Variable naming conventions

#Legal variable names:

myvar = "John" < ok

my_var = "John" < BETTER

_my_var = "John"

myVar = "John" < Classes

MYVAR = "John" < CONSTANTS

myvar2 = "John"

#Illegal variable names:

2myvar = "John"

my-var = "John"

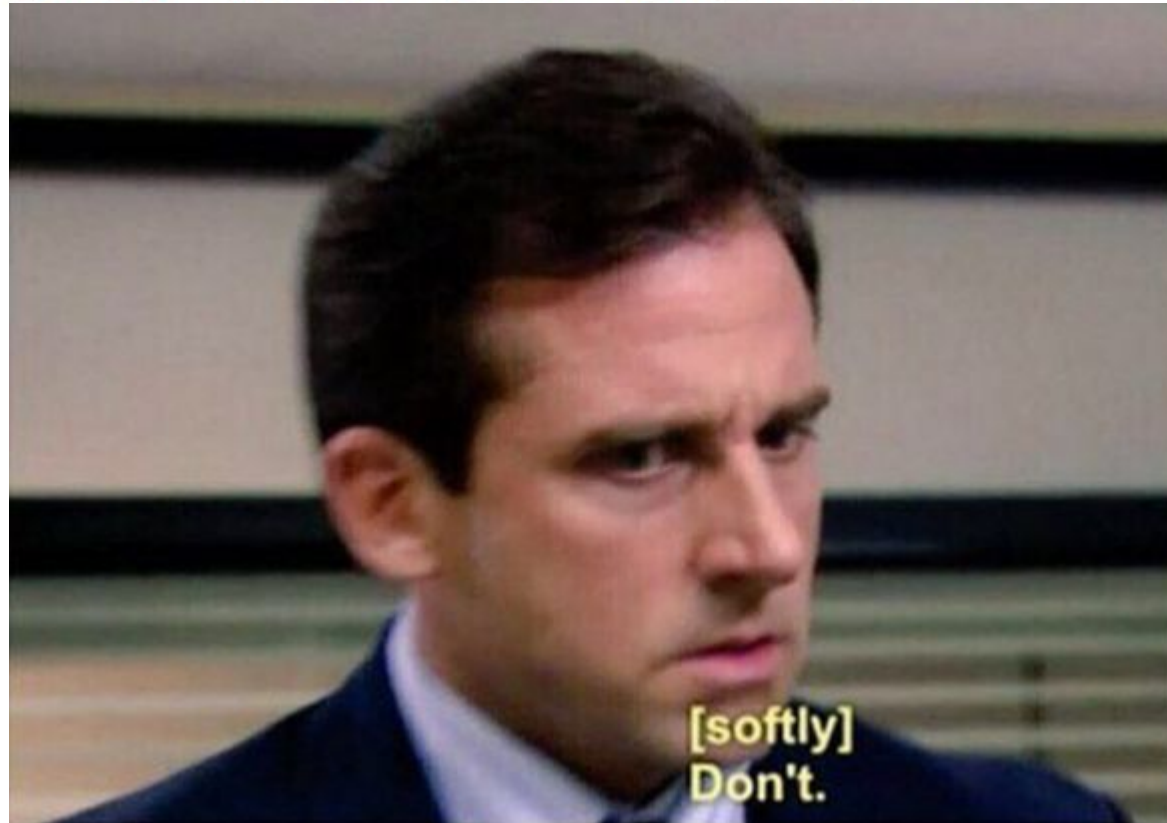
my var = "John"

RESERVED

and	except	lambda	with
as	finally	nonlocal	<u>while</u>
assert	false	None	yield
break	for	not	try
class	from	or	is
continue	global	pass	else
def	if / elif	raise	True
del	import	return	in

Variable naming conventions

`the_best_member_of_the_group_is_what_is_held_in_this_variable`



[softly]
Don't.

Variable naming conventions - others

Functions

- Snake Case

```
def process_data():  
    pass
```

Classes

- Camel Case

Model

Variable naming conventions

- Do not mix conventions
- Focus on clarity and descriptiveness
- Don't use it? Don't name it.

```
for _ in range(1000):  
    print('iteration')
```

```
def function_that_returns_a_lot():  
    return 1, 2, 3  
  
_, _, output = function_that_returns_a_lot()  
print(output)
```

Magic Numbers

```
def calculate_area(radius):  
    return 3.14 * radius ** 2
```

```
result = calculate_area(5)  
print(result) # Output: 78.5
```

```
pi = 3.14
```

```
    return pi * radius ** 2
```

```
result = calculate_area(5)  
print(result) # Output: 78.5
```

Magic Numbers

```
euros_count = 1000  
dollars_count = euros_count * 1.25 # 1250.0  
rubles_count = dollars_count * 60 # 75000.0
```

```
print(rubles_count)
```

```
dollars_per_euro = 1.25  
rubles_per_dollar = 60
```

```
euros_count = 1000  
dollars_count = euros_count * dollars_per_euro # 1250.0  
rubles_count = dollars_count * rubles_per_dollar # 75000.0
```

```
print(rubles_count)
```

Explicit Typing

- Type Hints
- Explicit Type

```
def calculate_area(radius: float) -> float:
    """
    Calculate the area of a circle given its radius.

    Parameters:
    radius (float): The radius of the circle.

    Returns:
    float: The area of the circle.
    """
    pi: float = math.pi
    return pi * radius ** 2
```

Explicit Typing - Type Hints

```
from typing import List

def record_telegrams(duration_minutes: int) -> List[str]:
    telegrams = []
    # Your implementation here
```

Comments and Docstrings

[PEP257 - Docstrings](#)

- Specific format
- Informative
- Auto import into documentation

```
def calculate_area(radius):  
    """  
    Calculate the area of a circle given its radius.  
  
    Parameters:  
    radius (float): The radius of the circle.  
  
    Returns:  
    float: The area of the circle.  
    """  
    pi = 3.14  
    return pi * radius ** 2  
  
result = calculate_area(5)  
print(result)  # Output: 78.5
```

The Art of Computer Programming

The Art of Computer Programming" is a comprehensive multi-volume work by Donald E. Knuth that serves as a foundational text in computer science.

- Commonly known as **TAOCP**
 - Algorithms
 - Data structures
 - Principles of computer programming.

The Art of Computer Programming

- Here are some key points about TAOCP:

1. **Authorship:** Donald Knuth, a renowned computer scientist and Stanford University professor, wrote the series. The first volume was originally published in 1968, and additional volumes have been released over the years.
2. **Algorithmic Concepts:** TAOCP delves deeply into the fundamental concepts of algorithms and their analysis. It explores various sorting and searching algorithms, as well as topics like combinatorial algorithms and arithmetic algorithms.
3. **Mathematical Rigor:** Knuth emphasizes mathematical rigor in the analysis of algorithms. The series includes mathematical proofs, equations, and discussions aimed at providing a thorough understanding of the algorithms presented.
4. **Typesetting System:** Knuth is also known for creating the typesetting system TeX, and he used it to typeset TAOCP. The books are recognized for their high-quality typesetting and the use of a special mathematical notation called "Literate Programming," where programs are embedded in a narrative text.
5. **Ongoing Work:** TAOCP is a work in progress, and Knuth has continually updated and expanded the series over the years. The series is planned to have seven volumes in total, with the author intending to cover a wide array of topics related to computer programming.
6. **Legacy:** The series has had a significant impact on the field of computer science and programming. It is considered a classic and has influenced generations of computer scientists and programmers.

Clean Code

- Clean Code: A Handbook of Agile Software Craftsmanship" is a book written by Robert C. Martin, also known as "Uncle Bob."

The first book on code I have encountered as a kid :D

Clean Code

- **Meaningful Names:** Use descriptive and meaningful names for variables, functions, and classes to enhance code readability.
- **Functions:** Keep functions short, focused on a single responsibility, and avoid side effects. Functions should do one thing and do it well.
- **Comments:** Write comments only when necessary, and make sure they add value. The goal is to make the code self-explanatory without relying on comments.
- **Formatting:** Maintain consistent and clean formatting to make the code visually appealing and easy to navigate.
- **Objects and Data Structures:** Hide implementation details behind abstractions and use proper data structures. Objects should hide their data and expose operations.

Clean Code

- **Error Handling:** Use exceptions rather than returning error codes, and handle errors at the appropriate level of abstraction.
- **Testing:** Write comprehensive unit tests to ensure the correctness of code. Aim for high test coverage to catch potential issues early.
- **Code Smells and Refactoring:** Learn to identify code smells (indicators of poor design) and apply refactoring techniques to improve code structure without changing its external behavior.
- **Concurrency:** Write code that is safe and easy to understand in concurrent environments.
- **Emergent Design:** Allow the design of the system to emerge based on the evolving understanding of the requirements. Avoid unnecessary complexity.

Clean Code

The overarching theme is that clean code is not only about aesthetics!

Creating code that is:

- Easy to work with
- Easy to understand
- Easy to modify

The book emphasizes the importance of craftsmanship in software development and provides practical advice on how to achieve it in the context of agile development practices.

The Power of Ten – NASA/JPL Coding Guidelines

1. *Restrict all code to very simple control flow constructs – do not use goto statements, setjmp or longjmp constructs, and direct or indirect recursion.*
2. *All loops must have a fixed upper-bound. It must be trivially possible for a checking tool to prove statically that a preset upper-bound on the number of iterations of a loop cannot be exceeded. If the loop-bound cannot be proven statically, the rule is considered violated.*
3. *Do not use dynamic memory allocation after initialization.*
4. *No function should be longer than what can be printed on a single sheet of paper in a standard reference format with one line per statement and one line per declaration. Typically, this means no more than about 60 lines of code per function.*
5. *The assertion density of the code should average to a minimum of two assertions per function. Assertions are used to check for anomalous conditions that should never happen in real-life executions. Assertions must always be side-effect free and should be defined as Boolean tests. When an assertion fails, an explicit recovery action must be taken, e.g., by returning an error condition to the caller of the function that executes the failing assertion. Any assertion for which a static checking tool can prove that it can never fail or never hold violates this rule (i.e., it is not possible to satisfy the rule by adding unhelpful “assert(true)” statements).*
6. *Data objects must be declared at the smallest possible level of scope.*
7. *The return value of non-void functions must be checked by each calling function, and the validity of parameters must be checked inside each function.*
8. *The use of the preprocessor must be limited to the inclusion of header files and simple macro definitions. Token pasting, variable argument lists (ellipses), and recursive macro calls are not allowed. All macros must expand into complete syntactic units. The use of conditional compilation directives is often also dubious, but cannot always be avoided. This means that there should rarely be justification for more than one or two conditional compilation directives even in large software development efforts, beyond the standard boilerplate that avoids multiple inclusion of the same header file. Each such use should be flagged by a tool-based checker and justified in the code.*
9. *The use of pointers should be restricted. Specifically, no more than one level of dereferencing is allowed. Pointer dereference operations may not be hidden in macro definitions or inside typedef declarations. Function pointers are not permitted.*
10. *All code must be compiled, from the first day of development, with all compiler warnings enabled at the compiler’s most pedantic setting. All code must compile with these setting without any warnings. All code must be checked daily with at least one, but preferably more than one, state-of-the-art static source code analyzer and should pass the analyses with zero warnings.*

The Power of Ten – NASA/JPL Coding Guidelines

1. Limit Line Length: Keep lines of code relatively short for readability.

Bad example

```
long_variable_name = calculate_some_really_long_function_name  
(argument1, argument2, argument3, argument4)
```

Good example

```
result = calculate_short_function(arg1, arg2)
```

The Power of Ten – NASA/JPL Coding Guidelines

2. Limit Function Length: Keep functions short and focused.

Bad example

```
def complex_function(arg1, arg2, arg3):  
    # 50 lines of code here  
    ...
```

Good example

```
def simple_function(arg1, arg2):  
    # A few lines of code here  
    ...
```


The Power of Ten – NASA/JPL Coding Guidelines

3. Limit File Length: Keep files reasonably small and focused.

Bad example:

LargeFile.py

Good example:

SmallFile1.py, SmallFile2.py, etc.

The Power of Ten – NASA/JPL Coding Guidelines

4. Limit Routine Length: Similar to function length, keep routines (methods) short.

Bad example

```
class SomeClass:  
    def long_routine(self, arg1, arg2):  
        # 50 lines of code here  
        ...
```

Good example

```
class SomeClass:  
    def short_routine(self, arg1, arg2):  
        # A few lines of code here  
        ...
```

The Power of Ten – NASA/JPL Coding Guidelines

5. Limit Variable Scope: Minimize the scope of variables.

Bad example

```
def some_function():  
    global some_var  
    some_var = 10  
    ...
```

Good example

```
def some_function():  
    some_var = 10  
    ...
```

The Power of Ten – NASA/JPL Coding Guidelines

6. Use Constants for Magic Numbers: Avoid magic numbers; use named constants.

Bad example

```
if x > 86400:
```

```
    ...
```

Good example

```
SECONDS_IN_DAY = 86400
```

```
if x > SECONDS_IN_DAY:
```

```
    ...
```

The Power of Ten – NASA/JPL Coding Guidelines

7. Limit Data Hiding: Don't use unnecessary access specifiers in classes.

Bad example

```
class SomeClass:
    def __init__(self):
        self.__private_var = 10
    def get_private_var(self):
        return self.__private_var
```

Good example

```
class SomeClass:
    def __init__(self):
        self.private_var = 10
```

The Power of Ten – NASA/JPL Coding Guidelines

8. Avoid Using Macros: In Python, this translates to avoiding global constants or variables.

Bad example

```
PI = 3.14159
```

Good example

```
import math  
pi = math.pi
```

The Power of Ten – NASA/JPL Coding Guidelines

9. Avoid Compound Statements: Keep statements simple and separate.

Bad example

```
if condition1 and condition2:
```

```
    ...
```

Good example

```
if condition1:
```

```
    if condition2:
```

```
        ...
```

The Power of Ten – NASA/JPL Coding Guidelines

10. Avoid Negative Logic: Write conditions in positive form when possible.

Bad example

if not is_invalid:

...

Good example

if is_valid:

...

General Rules

- First code for correctness – then for clarity
- Premature optimization is the root of all evil
- Comment the code! While Coding! (You will thank Yourself later)
- Readability becomes a priority AFTER the code works

- Code optimization

- 
1. Code works
 2. Clean the code
 3. Optimize running efficiency

- Make sure somebody else can use what You did!

Important Developer Acronyms

- PEP
 - TDD/BDD
 - CI/CD
 - ISTQB
 - SOLID
-
- DRY – Don't Repeat Yourself
 - KISS – Keep It Stupid Simple
 - YAGNI – You Ain't Gonna Need It
 - WYSIWYG – What You See is What You Get

Personal Favourites:

- PEBCAK – Problem Exists Between Chair and Keyboard
- PICNIC – Problem In Chair Not In Computer

S.O.L.I.D.

- **S**.ingle Responsibility
 - One class – one responsibility
- **O**.pen-Closed
 - Open for extension, Closed for Modification
- **L**.iskov Substitution
 - Replace parent with child – nothing breaks
- **I**.terface segregation
 - Do not depend on something you do not explicitly use
- **D**.ependency Inversion
 - Interfaces between classes and objects

Single Responsibility

- One class – one responsibility

```
class UserManager:
    new *
    def __init__(self):
        self.db = Database()

    new *
    def add_user(self, user):
        self.db.insert(user)

    new *
    def remove_user(self, user_id):
        self.db.delete(user_id)
```

```
class Database:
    1 usage new *
    def insert(self, user):
        # Logic to insert user into database

    1 usage new *
    def delete(self, user_id):
        # Logic to delete user from database
```

O.pen-Closed

- Open for extension, Closed for Modification

```
class Shape:
    new *
    def area(self):
        pass

new *
class Rectangle(Shape):
    new *
    def __init__(self, width, height):
        self.width = width
        self.height = height
    new *
    def area(self):
        return self.width * self.height
```

```
class Circle(Shape):
    new *
    def __init__(self, radius):
        self.radius = radius
    new *
    def area(self):
        return 3.14 * self.radius * self.radius
```

Liskov Substitution

- Replace parent with child – nothing breaks

```
class Bird:
    new *
    def fly(self):
        pass

new *
class Duck(Bird):
    new *
    def fly(self):
        print("Duck flying")
```

```
class Ostrich(Bird):
    new *
    def fly(self):
        raise NotImplementedError("Ostrich cannot fly")
```

Interface segregation

- Do not depend on something you do not explicitly use

```
class Printer:
    new *
    def print(self, document):
        pass
```

2 usages new *

```
class Scanner:
    new *
    def scan(self, document):
        pass
```

```
new *
class Photocopier(Printer, Scanner):
    pass
```

```
class MultiFunctionDevice(Printer, Scanner):
```

new *

```
def print(self, document):
    # Logic to print document
```

new *

```
def scan(self, document):
    # Logic to scan document
```

Dependency Inversion

- Interfaces between classes and objects

```
class Database:
    1 usage (1 dynamic)    new *
    def get_data(self):
        pass
```

```
class DataManager:
    new *
    def __init__(self, database):
        self.database = database
    new *
    def process_data(self):
        data = self.database.get_data()
        # Process data

db = Database()
data_manager = DataManager(db)
```



PEP – Python Enhancement Proposals

PEP8 - [Style Guide for Python Code](#)

PEP 20: The Zen of Python - A collection of guiding principles for writing computer programs in Python.

PEP 257: Docstring Conventions - Defines conventions for writing docstrings (documentation strings) for Python modules, functions, classes, and methods.

PEP 484: Type Hints - Introduces a syntax for annotating the types of variables, function arguments, and return values in Python code.

PEP 333: Python Web Server Gateway Interface (WSGI) - Defines a standard interface between web servers and Python web applications or frameworks.

PEP 3333: Python Web Server Gateway Interface v1.0.1 - An update to PEP 333 to clarify certain aspects and improve compatibility.

PEP 572: Assignment Expressions - Introduces the "walrus operator" (`:=`), allowing assignment expressions within other expressions.

PEP 498: Literal String Interpolation - Introduces f-strings, a more concise and readable way to format strings in Python.

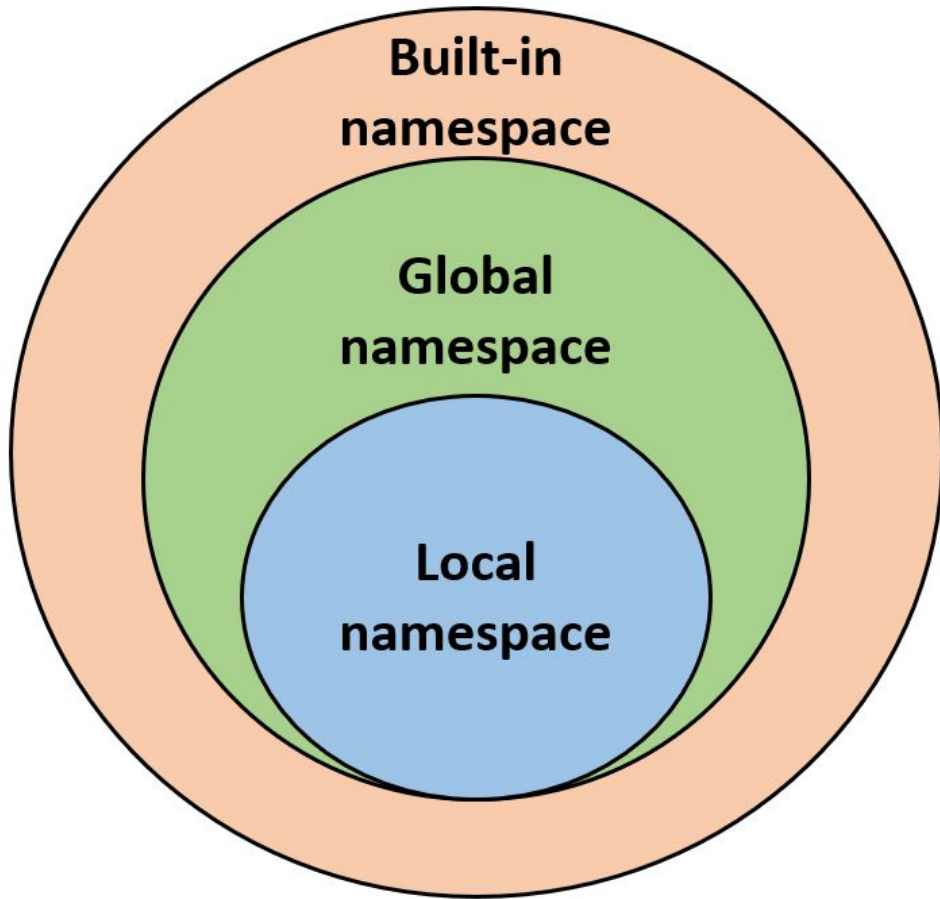
PEP 3107: Function Annotations - Introduces a syntax for adding metadata (annotations) to function parameters and return values.

PEP 394: The "python" Command on Unix-like Systems - Defines the expectations for the python command and its variants on Unix-like systems, particularly in relation to Python 2 and Python 3 coexistence.

Pre-commit and style check

- Pre-Commit
 - Requires configuration file (provided)
- Flake8
 - Can be installed into PyCharm as external tool
- Black
 - Executed from console, repairs all code in repo

Python Scopes



Type of Namespaces

```
namespace.py x
1 x = 10
2 y = 20
3
4
5 def outer():
6     z = 30
7
8     def inner():
9         x = 30
10        print(f'x is {x}')
11        print(f'z is {z}')
12        print(f'y is {y}')
13        print(len("abc"))
14
15    inner()
16
17 outer()
18
19 outer() > inner()
20
21 namespace x
22 /Users/pankaj/Documents/PycharmProjects/PythonTutorialPro/venv/bin/p
23 x is 30
24 z is 30
25 y is 20
26 3
27
28 Process finished with exit code 0
```

global

enclosed

local

local

built-in namespace has len() function

Thank You for Your attention!